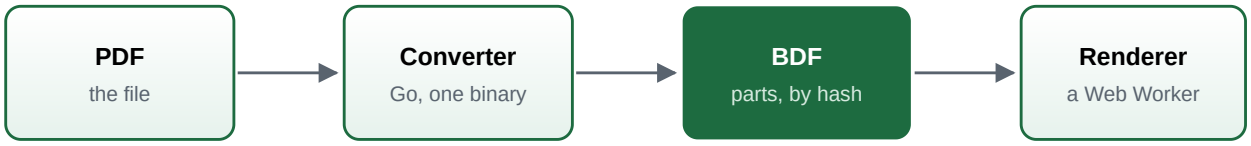
This is a PDF, drawn from BDF

The twenty pages you are reading were a PDF file. A converter written in Go turned them into BDF, and a small renderer in your browser draws them on a canvas.

What happened to this file



The browser fetches only the parts of the pages in view.

What is kept

- **Text.** Select it, copy it, search for it.
- **Fonts.** The glyphs in use, as WOFF2.
- **Vector graphics.** Zoom in: the lines stay sharp.
- **Links.** Like the repository, or the contents on page 7.
- **Structure.** Headings, lists and tables, for screen readers.
- **Metadata.** Title, author, dates.

> Try the search box above the pages: look for `canvas`, or for `フォント`. Below the pages, the same document is written back as a PDF file and as Markdown, and its text is handed to a search index.

What the browser already does

A PDF viewer written in JavaScript brings its own font rasterizer, image decoders and decompression. BDF is stored in the forms browsers take as they are, so its renderer has little left to do.

What	Stored as	Drawn with
Drawing instructions	One to one with Canvas 2D calls	`OffscreenCanvas` in a worker
Fonts	WOFF2, only the glyphs in use	`FontFace`
Images	JPEG, PNG, WebP, SVG …	`createImageBitmap`
Compression	Deflate, part by part	`DecompressionStream`
Encryption	AES-256-GCM, part by part	WebCrypto

日本語のページ

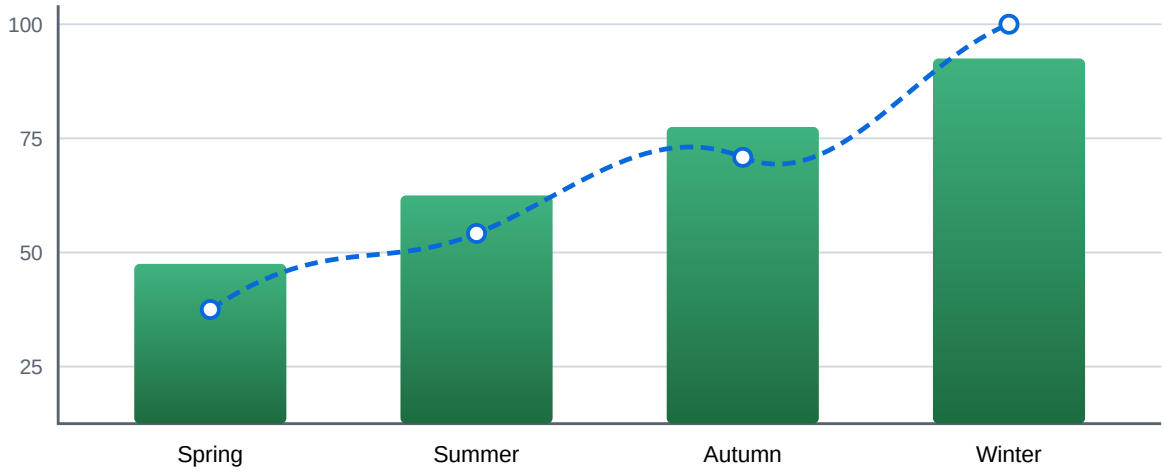
このページは PDF でした。Go で書かれた変換器が BDF に変換し、ブラウザの中の小さなレンダラが canvas に描いています。テキストは選択でき、検索もできます。「ふぉんと」とひらがなで検索しても「フォント」が見つかります。

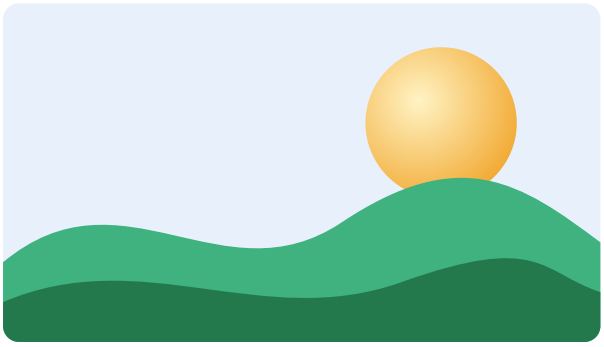
対象	格納する形	ブラウザの API
描画命令	Canvas 2D の呼び出しと 1 対 1	`OffscreenCanvas`
フォント	使う字だけの WOFF2	`FontFace`
画像	JPEG・PNG・WebP・SVG など	`createImageBitmap`

> BDF は文書を Part に分け、内容のハッシュで格納します。どのページも使うフォントは 1 度だけ保存され、ブラウザは表示するページに要る Part だけを取得します。

Shapes, gradients and a chart

Paths, clips, gradients and transparency are converted as they are. The chart below is vector art: its labels are text.





Other kinds of documents are converted into the same format: Word, Excel and PowerPoint files, diagrams, CAD drawings, books and scores.

A sheet of a workbook is not cut into pages: it scrolls, as in a spreadsheet.

Inside a BDF file

A BDF document is a manifest and a set of parts. The manifest is JSON: it names the views of the document, their pages, and the parts each page is drawn from. A part is a run of bytes stored under the hash of its content, so two pages that use the same font or the same picture share one part, and a part that did not change between two versions of a document is the same part.



fest gives the offset and length of every part. Each is compressed on its own, so a reader fetches one with a Range request and expands

The kinds of parts

Part	Holds	Who reads it
manifest	The views, their pages and layers, the metadata, the tables of data	The viewer, first of all
object	Drawing instructions of a page or a layer: paths, text, images, groups	The renderer, page by page
font	A font subset to the glyphs the document uses, as WOFF2	`FontFace`, once per font
image	A picture in the format it was in: JPEG, PNG, WebP, SVG	`createImageBitmap`
path, paint	A path or a gradient used by more than one object	The renderer, cached
text index	The text of a page, normalized, with the places of its characters	Search and text selection

Two shapes of the same document

A **single file** starts with the manifest and keeps every part after it, the fonts and the shared objects first, then the pages in order, so that the first page can be drawn before the rest has arrived: it is what a server streams as one download, or what a browser reads with Range requests, fetching the parts of the pages in view and nothing else. The **split form** writes each part as a file of its own, named by its hash, so that the parts can sit on object storage or a CDN as they are, cached forever, since a part never changes once it is named.

> The hash is also the name under which the browser caches a part. Two documents that share a font fetch it once.

Views: the shapes a document takes

A file is not always a stack of pages. A workbook is a set of sheets that scroll in both directions; a web page is one long column; a song is a card with a player on it. Each becomes a **view** of a document, and a document may hold several: a Word file gives its pages and, beside them, the same text as one column to scroll.

Kind	What it is	Made from	Shown as
`fixed`	Pages of a fixed size	PDF, slides, drawings, boards, scores, books	Pages that turn, or scroll
`flow`	Pages of text laid out for a paper size	Word, Markdown, HTML, EPUB	Pages, with a scroll view beside them
`scroll`	One column, as long as the text	Word, Markdown, HTML, EPUB	Scrolled, as a web page is
`sheet`	A grid of cells without page breaks	Excel, CSV, TSV, Parquet	Scrolled both ways, with frozen panes
`card`	One page that plays something	Audio files, scores	A card with a player

Layers and their roles

A page is drawn from one or more layers, and each layer may say what it is: the `background` of a slide, its `master`, a running `header` or `footer`. A writer of PDF files can leave a role out, to print on paper that already has the frame; a writer of Markdown leaves the running heads out by default, since a text has no pages.

- **A slide** is its master, its layout and its own content, three layers that the viewer flattens.
- **A sheet** is drawn in tiles as it is scrolled; its row and column headers, its grid and its frozen panes are described in the view, and the viewer draws them itself.
- **A board** of a circuit has a view for each of its layer files beside the views of its two faces, which the viewer shows as tabs.

Tabs, pages, tables

A document with several views shows them as tabs: the sheets of a workbook, the pages of a diagram, the model space and the layouts of a drawing. A view with pages has a page number; a sheet has none, and its text goes to a search index as one entry. Beside its drawing, a document can carry **tables** of data: the cells of a sheet, the values behind a chart, written out again as CSV, JSON or Parquet.

Text, as text

The glyphs on a page are drawn from the fonts of the document, but the text under them is kept as characters, in reading order, with the box of each one. That is what lets you select a sentence across a line break, copy it, and find it with the search box, and what a screen reader reads out: the headings, the items of a list, the cells of a table.

Text at 22 points, bold

Text at 16 points, regular

Text at 12 points, italic

Text at 10.5 points, in a serif face: Liberation Serif

`Text at 9 points, monospaced: Liberation Mono`

Text at 7 points, the small print: every glyph is still a glyph, and every character is still a character.

How search finds it

The text of a page is stored as it was written. When you search, it is normalized on the way to the comparison, and so are the words you typed; both sides take the same path, so what you type finds what is written, whichever way it was written:

- Compatibility forms are folded (Unicode NFKC): full-width `ＢＤＦ` finds `BDF`, and `①` finds `1`.
- Case is folded: `canvas` finds `Canvas` and `CANVAS`.
- Katakana is folded into hiragana: `ふぉんと` finds `フォント`.
- A word split by a line break, or by the end of a run, is still one word.

What the fonts keep

A font is embedded once, subset to the glyphs the document uses, as WOFF2. This document uses Liberation Sans, Liberation Serif, Liberation Mono and IPA Gothic; the three faces of Liberation Sans in use (regular, bold and italic) are three fonts. A PDF whose fonts are already subsets takes them along as they are, repacked as WOFF2; an Office document names its fonts, and the converter lays its text out with the fonts it finds, or with free ones that stand in for them.

> A run of text is written with the width the converter measured it at. The renderer draws it with the browser's own text drawing and, where the browser's width differs, scales the run to the width recorded, so lines neither overflow nor drift. Where the placement of each glyph matters, as in a kerned heading, the converter writes the glyphs one at a time.

Contents

Links into the document itself are kept as page links: click a line to turn to its page. Links to the web open in a new tab.

1 This is a PDF, drawn from BDF
2 What the browser already does
3 Shapes, gradients and a chart
4 Inside a BDF file
5 Views: the shapes a document takes
6 Text, as text
7 Contents
8 The formats the converters take
9 The formats, continued; and what comes out
10 Lines, curves and joins
11 Transparency, gradients and clips
12 変換の流れ
13 On a server, in Go
14 In the browser, in TypeScript
15 Search: from pages to an index
16 Forms: a filled template
17 Typography
18 Pages that turn, sheets that scroll
19 Passwords, protected mode and signatures
20 How this document was made

Elsewhere

- The site: the viewer, the forms, the thumbnails and the search text, each a page where files are converted inside the browser.
- The documentation: getting started, the formats, the sample architectures, the specification.
- The repository: Go packages, npm packages, and the bdf command.

How a link is kept

A link is a rectangle on the page with a target: an address on the web, or a page of this document. The converter reads the link annotations of a PDF, the hyperlinks of a Word file or a slide, and the cross-references of a book. The viewer draws nothing for them; it lays them over the page as real links, so the keyboard reaches them and a screen reader announces them.

> The entry of a table of contents is a link too, when the PDF has it as one: the destinations of a PDF, named or explicit, become page links.

The formats the converters take

Each format is a Go package of its own under `converter/`. A program links the ones it needs; the browser modules of the site are built the same way, one for PDF, one for Office files, one for web documents and one for images. The table runs on to the next page.

Format	Files	What comes out
PDF	.pdf	Pages with their fonts, text, links, and the structure of a tagged PDF
Illustrator	.ai	Each artboard a page; the PDF inside the file is what is read
Word	.docx .docm .dotx .dotm	Pages, and one long column to scroll; vertical Japanese text too
PowerPoint	.pptx .pptm .ppsx .potx	Slides with their masters, charts, SmartArt and formulas
Excel	.xlsx .xlsm .xltx	Sheets that scroll, with frozen panes, headers and tabs; the cells as tables
CSV, TSV	.csv .tsv .tab	A sheet; the encoding and the delimiter are detected
Parquet	.parquet	A sheet of the first rows, with the schema
Visio	.vsdx .vsdm .vstx	Each page a page, with its shapes and connectors
draw.io	.drawio .dio, .png .svg exports	Pages as tabs, with the icon sets of today
Windows metafile	.emf .wmf	The drawing as a page
DXF	.dxf	Model space and the layouts, each a view
Jw_cad	.jww	The drawing, with its layers and its text
SXF	.p21 .p2z .sfc	The drawing of the Japanese public-works exchange format
CGM	.cgm .cgz	Binary and clear-text metafiles
HP-GL/2	.plt .hpgl .rtl	A plot with its pens; HP RTL raster too
Gerber, Excellon	.gbr .gtl .drl … .zip	The top and the bottom of a board, and a view for each layer file
KiCad	.kicad_sch .kicad_pcb .zip	Schematic sheets and the board, front and back

The extensions listed are the common ones; the converters look at the content of a file first and at its name second, so a file with the wrong extension still converts.

The formats, continued

Format	Files	What comes out
InDesign	.idml	The pages of a layout, with its text frames and its spreads
HTML	.html .htm .xhtml .mhtml	The article of a page in reader mode, as a column or as pages; MathML formulas
Markdown	.md .markdown .mdx	CommonMark and GitHub Flavored Markdown, with its tables and formulas
EPUB	.epub	Reflowable books as pages that turn, vertical Japanese too; fixed layouts as pictures
MML	.mml	Music Macro Language, engraved as a score the viewer plays
MIDI	.mid .midi .kar	A Standard MIDI File as a score, with the words of a karaoke file
MusicXML	.musicxml .mxl	A score, engraved with the glyphs of Bravura
Audio	.mp3 .m4a .flac .ogg .wav .aiff	A card with the cover art, the tags and the lyrics, which the viewer plays
Fonts	.ttf .otf .ttc .woff .woff2	A specimen: the glyphs, the features, the sizes
Images	.png .jpg .gif .webp .avif .bmp .svg	The image as a page, with its EXIF data
TIFF	.tif .tiff	Each image of the file a page
Photoshop	.psd .psb	The composite image, or each artboard a page

And what comes out

A document is read back as easily as it is drawn. The same Go packages that a server links to convert a file write it out again in the forms a system around it asks for:

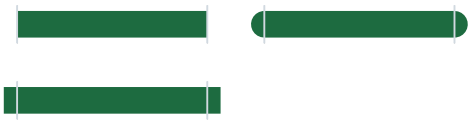
- **A PDF file**, with the fonts of the document embedded, plain or tagged, or as PDF/A for keeping.
- **Markdown**, with the pictures as files beside the text, embedded in it, or left out, and the tables inline.
- **The text of each page** with the metadata of the document, as JSON, for a search index.
- **Thumbnails** drawn without a browser, by a rasterizer in pure Go: PNG, JPEG or WebP.
- **A page as SVG**, with its fonts beside it; or as a raster at any resolution, for printing.
- **Tables** the document holds beside its drawing, as CSV, JSON or Parquet.

> The site shows three of these under the pages: this document as a PDF file, as Markdown, and as search text, each made inside your browser.

Lines, curves and joins

A stroke keeps its width, its caps, its joins and its dashes. The instructions below are the ones Canvas 2D has, which is why they survive the trip: a line cap in the PDF is a line cap in the file, and a line cap on the canvas.

Caps: butt, round, square



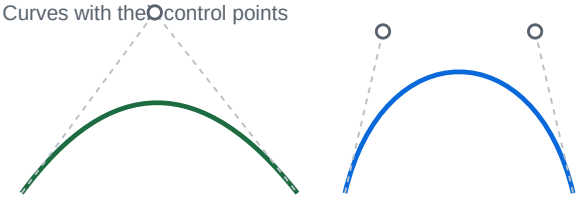
Joins: miter, round, bevel



Dashes



Curves with their control points



Polygons and fill rules



Widths



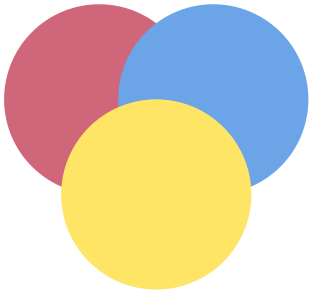
0.5, 1, 2, 4 and 8 units wide: zoom in, and they stay sharp.

> The converter writes a path once and refers to it by hash when the same shape repeats: the dots of a chart, the symbols of a schematic, the pads of a board.

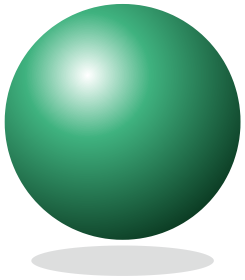
Transparency, gradients and clips

Alpha is kept per fill and per group; a group with an opacity of its own is drawn to a layer and composited, as Canvas 2D does. Gradients keep their stops, and a clip is a path like any other.

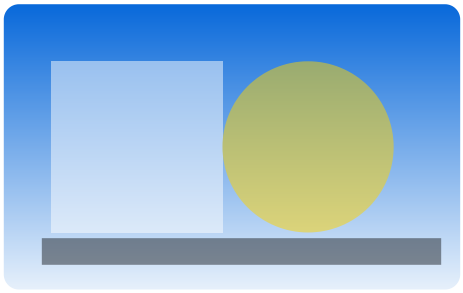
Three fills at 60% opacity



A radial gradient



A group at 50%, over a gradient



Six stops



A clip in the shape of a word



#1d6b40	#3fb27f	#0969da	#f0a020	#b00020	#59636e

The colors of this document, as the swatches a designer would keep. A fill is written as sRGB with an alpha; a PDF in CMYK or a spot color is converted to it.

Blend modes beyond the normal one (multiply, screen, and the others of Canvas 2D) are kept by name, and so are soft masks, as a group drawn to a layer with its alpha taken from the luminosity of another.

変換の流れ

BDF への変換には 2 つの道があります。どちらも同じ Go のパッケージを使い、同じ Part を作り、同じレンダラが描きます。違うのは、変換がどこで走るかだけです。

サーバーで変換する

文書を受け取ったサーバーが変換し、BDF を 1 つのファイルとして保存します。ブラウザは Range リクエストで、表示するページの Part だけを取り寄せます。サーバーは文書を持っているあいだに、サムネイルと検索用のテキストも作れます。一度変換した文書は何度でも開けます。

ブラウザで変換する

同じ変換器を WebAssembly にビルドし、Web Worker の中で動かします。開いたファイルはブラウザの外に出ません。このサイトのビューアがそうしています。PDF は 1 ページずつ変換され、見えているページから順に描かれます。

観点	サーバーで	ブラウザで
向いている文書	何度も開かれるもの、検索したいもの	大量に作られるが、まれにしか開かれないもの
必要なもの	Go のサーバー 1 つ（外部プログラムは不要）	wasm モジュールと、フォント
サムネイル	純粋な Go のラスタライザで描く	同じラスタライザを wasm で動かす
検索	ページごとのテキストを検索エンジンへ	ビューアの検索（この文書の検索ボックス）

何が変わらないか

- 描画の結果。同じ Part を同じレンダラが描くので、どちらの道でも同じ見た目になります。
- テキスト。選択、コピー、検索、読み上げは、形式によらず同じように動きます。
- フォント。文書が使う字だけを WOFF2 にして、一度だけ保存します。

> 「ぶらうざ」とひらがなで検索しても、このページの「ブラウザ」が見つかります。検索の前に、文書の側も入力の側も同じ正規化を通るからです。

On a server, in Go

A server converts a file with one call, and keeps the document for what it needs next: the file to send to browsers, a thumbnail for a list, and the text of each page for a search engine. No office suite runs, and nothing is spawned: the converters are packages in the server's own binary.

```
import (
        "log"
        "os"

        "github.com/shibukawa/bdf/converter"
        _ "github.com/shibukawa/bdf/converter/all" // every format; or link only the ones
        "github.com/shibukawa/bdf/thumbnail"
)

func ingest(path string) error {
        res, err := converter.ConvertFile(path, "", &converter.Options{})
        if err != nil {
                return err
        }
        log.Printf("%s: %d warning(s)", path, len(res.Warnings))

        // the document as a single file: streamed whole, or read with Range requests
        out, err := os.Create(path + ".bdf")
        if err != nil {
                return err
        }
        defer out.Close()
        if err := res.Doc.WriteSingle(out); err != nil {
                return err
        }

        // a thumbnail of the first page, drawn without a browser
        th, err := thumbnail.Make(res.Doc, &thumbnail.Options{Size: 256})
        if err != nil {
                return err
        }
        png, _ := os.Create(path + ".png")
        defer png.Close()
        thumbnail.Encode(png, th.Image, thumbnail.PNG)

        // the text of every page, with the metadata, for the search index
        text, err := res.Doc.SearchText()
        if err != nil {
                return err
        }
        return index(path, text) // text.Meta, text.Views[i].Pages[j].Text
}
```

The same three steps are the `bdf` command's `generate`, `thumbnail` and `text`. A document already converted is read back with `bdf.OpenSingleFile`, so its text can be indexed again without converting it again.

In the browser, in TypeScript

The renderer is an npm package of about 23 KB gzipped. It runs in a Web Worker, which fetches the parts, draws the pages on an `OffscreenCanvas` and hands bitmaps to the page. The main thread places them, lays the text over them, and asks the worker to search.

```
import { BdfWorkerClient, buildTextLayer } from "@bdfkit/render";

const client = new BdfWorkerClient(new Worker(new URL("worker.js", import.meta.url), { typ

// a single file on a server: the pages are fetched by Range requests as they are shown
const manifest = await client.open({ kind: "single", url: "demo.bdf", range: true });
const view = manifest.views[0];

// a page as a bitmap, at the scale of the screen
const bitmap = await client.page(view.id, 0, window.devicePixelRatio);
canvas.getContext("2d").drawImage(bitmap, 0, 0);

// its text, laid over the bitmap as real text: selection, copy, find in page, screen read
const content = await client.content(view.id, 0);
pageElement.append(buildTextLayer(content, scale));

// search the whole view, then the boxes of the hits, to mark them on the pages
const hits = await client.search(view.id, "canvas", { limit: 500 });
const boxes = await client.locate(view.id, hits);
```

A page at a time

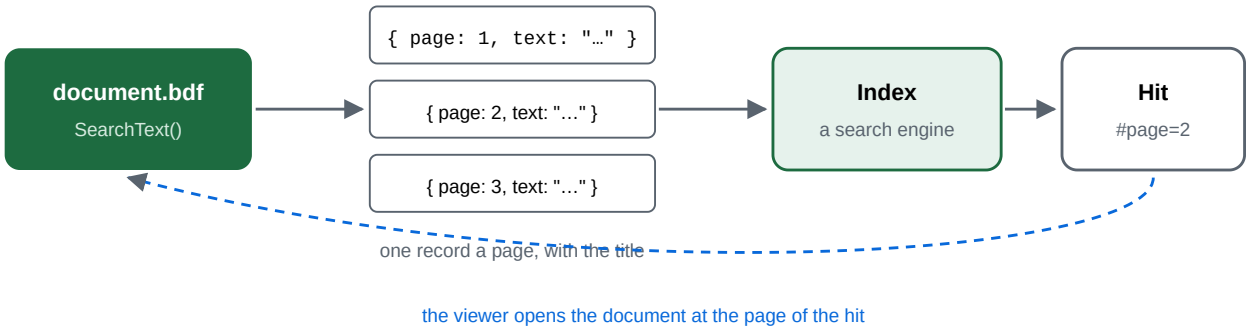
A file opened on the page goes to a second worker, the converter, built from the Go packages as WebAssembly. A PDF comes back as an outline first, every page sized, and then page by page, the pages in view first; the viewer swaps in the finished document once every page is done. An Office file comes back whole.

```
const opened = await converter.open("lib/bdf-pdf.wasm", file); // the outline: every page
const doc = await BdfDocument.open(new BufferSource(opened.bdf));
for (const i of pagesInView) {
  const page = await converter.page(opened.stream, i);           // one page, with the par
  doc.addPage(view.id, i, await BdfDocument.open(new BufferSource(page.bdf)));
}
const whole = await converter.finish(opened.stream);             // the rest, and the docu
```

The site's viewer, forms, thumbnails and search text pages are built on these two workers and nothing else.

Search: from pages to an index

Search needs the text of a document, not its drawing. `SearchText` gives the metadata of the document and the text of each page, a sheet counting as one entry, and each page becomes one record of the index: an id, the file, the view, the page number, the title, the text. A hit knows its page, so a result opens the document there.



What goes to the engine

- **The text of a page**, in reading order, with its line breaks; the engine normalizes it its own way, as the viewer's search does.
- **The title** of the document, from its metadata (Dublin Core: title, creator, dates, language), so that a hit can be listed by name.
- **The place**: the file, the view and the page, so that a hit can be opened where it was found.

Two engines

The sample architecture *search* sends the records to Meilisearch from a Go server and keeps the API key there: the browser queries the server, which proxies the query and returns the hits with a highlighted snippet. On the site, below these pages, the same records are indexed inside your browser by MiniSearch, a JavaScript library, with Japanese text as pairs of neighboring characters, the usual way to index it without a dictionary. Either way, the engine never sees the drawing, and the document is converted once.

> A sheet has no page number: its text is one entry, and a hit opens the sheet. A password-protected document gives its text only while the server holds the password.

Forms: a filled template

A form is a Word, Excel or PowerPoint file with tags in its cells and paragraphs, or a PDF with form fields. The tags say what the data looks like: a text, a number, a date, a picture, a bar code, or rows that continue onto further pages. Filled with data, the template becomes a document like this page, which was made of a table of items and a few values.

INVOICE

No. INV-2026-0042 · Issued 2026-10-26 · Due 2026-11-25

Example Trading Co.
1-2-3 Sample Street, Suite 400
Sampleville 100-0001
billing@example.com

Bill to
Placeholder Industries Ltd.
Attn: Accounts payable

#	Item	Qty	Unit price	Amount
1	Document conversion, server license (annual)	1	1,200.00	1,200.00
2	Thumbnail service, per 10,000 documents	12	35.00	420.00
3	Search index export, per 10,000 pages	8	18.50	148.00
4	Print server, per queue	3	60.00	180.00
5	Support, priority (quarterly)	4	250.00	1,000.00
			Subtotal	**2,948.00**
			Tax (10%)	**294.80**
			Total	**3,242.80**

Payment by bank transfer to the account on the attached sheet. Sample figures: this invoice is an example and bills no one.

What the template said

The items were a row of tags, `{{#items}}` … `{{/items}}`, which repeats for each item of the data and carries on to a second page when it runs out of room, with the subtotal of each page carried forward. The totals were expressions over the rows. A JSON Schema of the data is written out of the template, so that the form which collects it can be made of the schema alone.

> Filled forms are written as PDF files, tagged or as PDF/A, and can be signed: a signature field on a page, and a manifest digest that shows whether anything changed since.

Typography

Every face of a font is a font of its own in the file: the regular, the bold and the italic of Liberation Sans are three subsets, each holding the glyphs that face uses and no others. A drop capital in Liberation Serif, as at the start of this paragraph, adds a fourth, with one glyph in it. The space between letters and between lines is not a property of the text but of its position: each run of text is placed where the converter put it, at the width it measured.

Text set in two columns stays two columns of text. The reading order of the characters is the order of the columns, so a selection that starts in the first column and ends in the second takes the end of the one and the start of the other, as it should, and a search for a word that wraps from the bottom of a column to the top of the next still finds it.

Superscripts and subscripts are small glyphs placed higher or lower: H_2O, x^2, a footnote mark[1]. Ligatures such as fi and fl in a serif face are single glyphs that the text under them keeps as two characters, so that a search for *file* finds *file*.

A paragraph in Liberation Serif, a face with the metrics of Times New Roman. It reads differently, and it is a different subset. Its italic, *like this*, is another, and its bold, **like this**, a fourth.

```
Monospaced text, in Liberation
Mono: the cells of a code listing
line up, and a tab is as wide as
the converter made it.
```

> *The glyphs of a page are the shapes a reader sees; the characters are what the reader means. A document format for browsers keeps both, and keeps them apart.*

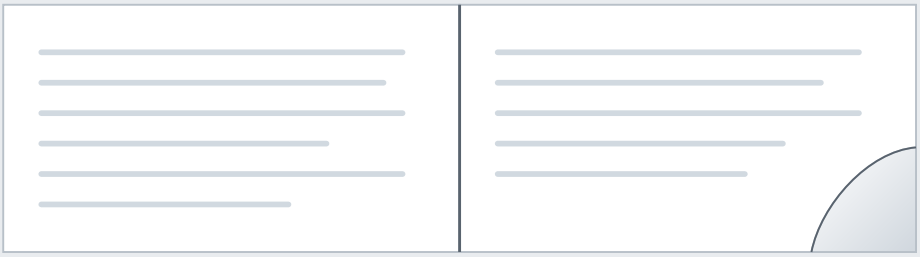
“A character stays a character, whatever glyph draws it.”

Letter-spacing, l i k e t h i s , and word-spacing are positions too. Small text, , is drawn from the same font at a smaller size. Text in color, like this, and highlighted text, are a fill and a rectangle behind it.

[1] A footnote is a paragraph like any other, at the foot of the page; its mark is a link in a PDF that has one, and plain text in one that has not.

Pages that turn, sheets that scroll

The viewer on the site shows this document as a book: two facing pages, the first one alone like a cover, turned by the buttons, the keys, a tap, or a pull on a corner. The menu above the pages shows it one page at a time instead, or as pages to scroll through. On a phone, where two pages would be too small, it opens one at a time by itself.



page 18 page 19 — pull the corner

Keys

`→` `←` turn the pages

`Home` `End` the first and the last

`Ctrl` + `F` search the whole document

`Enter` `Shift` + `Enter` the next and the previous match

`Esc` leave the search

Layouts

- **Two pages**: a spread, the first page alone. Pages are drawn at the size they fit.
- **One page**: each page as large as the stage allows.
- **Scroll**: the pages one above the other, as wide as the stage; each is drawn as it comes near.

A sheet is not a page

A workbook opens on its first sheet, with the tabs of the others. The sheet scrolls in both directions and is drawn in chunks as it is scrolled; its frozen rows and columns stay where they are. A search finds a cell and scrolls to it. The same holds for a CSV file or a Parquet file: a sheet of their rows, with the first rows of a large file and the schema of a Parquet file.

> The page links of this document (the contents, say) turn the book to the page; in the scrolled layout they scroll to it.

Passwords, protected mode and signatures

A document can be encrypted part by part with AES-256-GCM, under a content key made for that document. The key is wrapped for each reader: for a password, with a key derived from it by PBKDF2 (600,000 rounds of SHA-256); for a key pair, with ECDH. The manifest of the document is sealed too; only an outer manifest stays in the clear, naming the cipher, the key slots and the sealed parts, so that the file can be split and joined without the key. The browser unwraps the content key with WebCrypto and decrypts each part as it fetches it; a wrong password fails to unwrap the key, and the document is never read with it.

Protected mode

A server that must not hand the whole document to a reader can hand out **segments**: a few pages at a time, each sealed for one reader with a key agreed for that request alone, so that a segment saved from the browser opens for no one else and cannot be put back together into the document. Search runs on the server, which holds the document, and returns only the places of the hits. The viewer of the site has a mode for it, with an example server beside it.

Mode	The browser gets	Can save	Search
Plain	The document, part by part or whole	The file	In the browser
Encrypted	The document, and decrypts it with the password	The file, still encrypted	In the browser, once opened
Protected	Segments of a few pages, each sealed for this reader	Nothing that opens again	On the server, by places

Signatures and tamper evidence

A document can carry a digest of its manifest and of every part, signed with a key, so that a reader can tell whether anything changed since it was signed. A filled form can be signed field by field: each signature field on its page holds a picture of the signature and the signature of the values beside it, and a party that signs its half of a contract leaves the other half open for the other party. Written as a PDF file, the signatures become PDF signatures (PAdES), which other readers verify too.

- **Manifest digest**: the hash of the manifest in its canonical form (JCS), over the hashes of the parts.
- **Value signatures**: the values of a form, signed with the key of the person who filled them in.
- **Seals**: a picture of a seal or a signature, placed in its field, drawn as part of the page.

> A password-protected PDF or Office file is converted with its password, and the document that comes out is plain: a server that keeps it decides for itself whether to encrypt it again, and with what.

How this document was made

This document is an HTML file, printed to a PDF by headless Chromium with its fonts embedded, and converted into BDF by the `bdf` command when the site is built. The PDF is 20 pages of A4; the BDF holds the same pages, their text, their links and the structure of the tagged PDF, with the fonts subset to the glyphs in use.

Step	Tool	What it makes
1	`site/demo/demo.html`	The pages, as HTML and SVG, with free fonts: Liberation and IPA
2	Chromium, `page.pdf()`	`demo.pdf`: a tagged PDF with the fonts embedded
3	`bdf generate demo.pdf demo.bdf`	The document shown on the site, as a single file
4	The viewer, in a worker	The pages drawn on a canvas, fetched by Range requests as they are shown
5	`bdf pdf`, `bdf markdown`, `bdf text`	Below the pages on the site: the way back out, inside the browser

Metadata

Title	This is a PDF, drawn from BDF
Author	BDF
Language	English, with pages in Japanese
Pages	20, A4 (210 × 297 mm)
Fonts	Liberation Sans, Liberation Serif, Liberation Mono (SIL Open Font License); IPAPGothic (IPA Font License)

What to try next

- Search the document for `canvas`, `フォント`, or `ぶらうざ`, and watch the hits on the pages.
- Zoom in on the strokes of page 10: they are vector art, and stay sharp.
- Write the document back out as a PDF, as Markdown, or as search text, under the pages on the site, and search it with MiniSearch.
- Drop a file of your own on the viewer: it is converted inside your browser and never uploaded.

> The end. Back to the first page. The text of this document is MIT-licensed, with the repository.